

Criptoanálisis Distribuido y Hijacking de Rutas RIP

Integración de seguridad informática en redes y sistemas de software (Gpo 403)
TC2007B

Santiago Hernández
A01787550

3 de octubre de 2026

1 Introducción

RIPv2 transporta actualizaciones con autenticación opcional. Cuando se habilita, el resumen criptográfico con clave (*keyed message digest*, RFC 2082) es lo único que impide inyectar una actualización falsificada. Un atacante que produzca una actualización con un digest válido puede inyectar / sobrescribir rutas y desviar el tráfico hacia nodos comprometidos.

Este reporte reconstruye el digest (Sección 2) y, a partir de eso, recupera por fuerza bruta las claves compartidas de seis capturas RIPv2 autenticadas. La Sección 3 describe el protocolo de coordinación TCP entre maestro y trabajadores; la Sección 4, la partición del espacio de claves; la Sección 5, la tolerancia a caídas de nodos; la Sección 6, el rendimiento medido; y la Sección 7 cierra con conclusiones y trabajo diferido.

2 Ingeniería inversa de la autenticación MD5 de RIP

Cada captura es una Respuesta RIPv2 enviada a la dirección multicast 224.0.0.9 sobre UDP/520. La autenticación de tipo 3 coloca una cabecera de auth como primera entrada de la tabla de rutas y añade un tráiler de auth que transporta el digest (16 bytes). La captura se descompone como en la Tabla 1, ese digest de 16 bytes es el campo que el ataque debe reproducir.

Región	**Bytes**	**Contenido (captura tamaño 06)**
Cabecera RIP	4	command=2, version=2, routing-domain=0
Cabecera auth (RTE)	20	AFI=0xFFFF, auth-type=3, pkt-len=44, key-id=1, auth-len=20, seq, reservado
Entrada de ruta	20	la ruta anunciada
Tráiler de auth	20	AFI=0xFFFF, type=0x0001, seguido del digest de 16 bytes

Tabla 1: Formato del paquete autenticado RIPv2/RFC 2082 (payload de 64 bytes).

Construcción del digest (hallazgo principal). El digest capturado corresponde a keyed-MD5 de RFC 2082. El campo de digest de 16 bytes se sustituye primero por la clave y se calcula un MD5 simple sobre todo el paquete. Sea D la región desde la cabecera RIP hasta los campos `AFI/type` del tráiler (todo salvo los 16 bytes del digest) y K_{16} la clave ASCII rellenada con bytes 0-16, como en la ecuación (2.1).

$$\text{digest} \;=\; \text{MD5}\big(\, D \,\|\, K_{16} \,\big). \tag{2.1}$$

Se probaron ambas construcciones (keyed-MD5 y HMAC-MD5), solo la ecuación (2.1) sobre un alfabeto base-36 (letras minúsculas y dígitos, no hexadecimal) reprodujo el digest capturado, el espacio hexadecimal se agotó sin coincidencia (keyed-MD5 en 7.0 s, HMAC-MD5 en 20.4 s). Por lo tanto, base-36 con keyed-MD5 es la construcción confirmada.

Claves recuperadas y validación. Se recuperó una clave por captura, pero ninguna se aceptó por coincidir con un solo paquete. Se recalculó el digest keyed-MD5 para todos los paquetes de su captura. Las seis claves reproducen el digest en el 100% de los paquetes (179/179 en total), lo que descarta coincidencias. La Tabla 2 resume cada clave, su índice base-36 y la validación por captura.

Captura	Long.	Espacio 36^n	Clave	Índice base-36	Valid.
06_01	6	2,176,782,336	`ykusrq`	1,618,816,210	21/21
06_02	6	2,176,782,336	`miy4or`	1,673,128,894	57/57
07_01	7	78,364,164,096	`vs7evd8`	18,253,050,367	27/27
07_02	7	78,364,164,096	`tpx6pje`	31,666,124,081	24/24
07_03	7	78,364,164,096	`nh3cote`	32,269,346,987	28/28
08_01	8	2,821,109,907,456	`n3s4mmcv`	2,456,777,905,475	22/22

Tabla 2: Claves recuperadas (todas base-36, keyed-MD5) y validación contra los paquetes autenticados de cada captura.

Implicaciones de seguridad. keyed-MD5 frente a HMAC cambia cómo se calcula un candidato, no cuántos existen. la debilidad es el tamaño del espacio, no la elección del MAC. Para seis caracteres base-36 el espacio es $36^6 = 2{,}176{,}782{,}336$ candidatos, cae en minutos. Un atacante que capture un solo paquete autenticado puede recuperar la clave sin conexión, de modo que el digest ofrece poca protección con claves cortas sobre un alfabeto pequeño. Cada carácter adicional multiplica el espacio por 36, la clave de ocho caracteres demando días de cómputo distribuido (Sección 6).

3 Protocolo de coordinación TCP y tipos de mensaje

El main y los trabajadores se coordinan sobre TCP con un protocolo orientado a líneas. Cada mensaje es un objeto JSON terminado por salto de línea (NDJSON), de modo que el tráfico resulta legible al depurar. El reparto es por demanda (*work-pulling*), el main mantiene una cola de bloques y entrega uno cada vez que un trabajador lo solicita (un nodo más rápido pide más seguido), y el rendimiento global no queda limitado por el nodo más lento. La Tabla 3 lista los tipos de mensaje.

Mensaje	Dirección	Propósito / campos
`HELLO`	W→M	registro; id de worker, host, pid, núcleos
`PARAMS`	M→W	datos del objetivo, digest, alfabeto, longitud, construcción
`REQUEST`	W→M	solicita un bloque del espacio de claves
`ASSIGN`	M→W	{id de bloque, inicio, fin}
`PROGRESS`	W→M	{bloque, probadas, tasa}; también sirve de heartbeat
`HEARTBEAT`	W→M	señal de vida mientras está sin trabajo entre bloques
`DONE`	W→M	bloque recorrido por completo sin coincidencia
`FOUND`	W→M	{bloque, clave, índice} al hallar una coincidencia
`NOWORK`	M→W	cola momentáneamente vacía; reintentar
`STOP`	M→W	alto global (clave hallada o espacio agotado)

Tabla 3: Protocolo de mensajes maestro–trabajador.

Ciclo de vida. Un trabajador envía `HELLO`, recibe `PARAMS` y alterna `REQUEST` y `ASSIGN`, procesando cada bloque en sublotes mientras emite `PROGRESS`, y reporta `DONE` o `FOUND`. Ante un `FOUND`, el main vuelve a calcular el digest de forma independiente antes de confiar en la clave y solo entonces manda `STOP` para que ningún nodo siga hasheando. El maestro lanza subprocesos trabajadores locales que se conectan por loopback y crackean junto a los remotos. La reverificación más el `STOP` garantizan un único resultado confiable.

4 Estrategia de partición del espacio de claves

El espacio de búsqueda es el conjunto de todas las cadenas de longitud fija sobre el alfabeto base-36. Para seis caracteres su tamaño es $N = 36^6 = 2{,}176{,}782{,}336$. Cada índice entero $i \in [0, N)$ se convierte determinísticamente en una clave mediante codificación posicional, por lo que un intervalo de índices corresponde a un conjunto de claves distintas y dos intervalos disjuntos no se traslapan. El main divide $[0, N)$ en bloques de tamaño fijo C, obteniendo $\lceil N/C \rceil$ bloques (por ejemplo, $C = 5 \times 10^6$ da 436 bloques) servidos desde una queue.

La asignación es dinámica. El main conserva una queue de bloques pendientes y un registro de los asignados (*inflight*). Ante cada `REQUEST`, si hay pendientes, entrega el siguiente mediante `ASSIGN`$(c,\ cC,\ \min((c{+}1)C, N))$ y lo marca como asignado. Si no quedan pendientes pero sí asignados, responde `NOWORK` porque alguno podría reencolarse, y si no queda ninguno, marca el espacio como agotado y difunde `STOP`. Un bloque solo se da por terminado cuando llega su `DONE`.

5 Detección de caída de nodos y reasignación

Ningún bloque asignado se pierde en silencio, el riesgo es que un bloque perdido haga terminar la búsqueda informando "no encontrada" tras saltarse la clave. El main se defiende de dos modos de fallo y, en ambos, reencola el bloque al frente de la cola de pendientes para que otro trabajador lo retome.

El primero es la caída dura, `kill`, el socket TCP se cierra, la lectura del maestro devuelve EOF y el manejador reencola todos los bloques que tuviera ese trabajador. El segundo es la partición de red: el socket sigue abierto pero paran los heartbeats de `PROGRESS`; un hilo reencola cualquier bloque cuyo último heartbeat supere un timeout T (5 s por omisión) y marca el nodo como muerto.

6 Métricas de rendimiento

Todas las mediciones usan el digest de tamaño 06 y la construcción keyed-MD5. El host tiene 10 núcleos lógicos, las tasas se expresan en millones de claves hasheadas por segundo (Mkeys/s). Se tomaron tres mediciones: BM1 (un solo núcleo), BM2 (escalamiento por número de procesos) y BM3 (ejecución distribuida instrumentada sobre sockets). La Tabla 4 las resume; el barrido completo de BM2 está en el Apéndice A.

Medición	**Agregado**	**Por nodo/núcleo**
BM1 un núcleo (base)	2.55	2.55
BM2 escalamiento, 8 procesos	10.67	1.33
BM2 escalamiento, 10 procesos	10.95	1.09
BM3 distribuido, 8 trabajadores TCP	12.56	≈1.73

Tabla 4: Rendimiento de hashing (Mkeys/s). Barrido completo en `results/benchmarks.txt`.

CPU-bound vs. network-bound. En la corrida distribuida el tiempo de cómputo agregado fue 69.1 s frente a 0.03 s de espera en sockets, es decir, 99.96% del tiempo dedicado a hashear. El rendimiento escala con los núcleos hasta saturar el host, la tasa por núcleo cae cuando el número de procesos supera el presupuesto de núcleos físicos (Tabla 4 y Apéndice A).

Tiempo de recuperación extremo a extremo. A la tasa de un núcleo (2.55 Mkeys/s), alcanzar el índice de la clave de tamaño 06 `ykusrq` (1.62×10^9) toma del orden de 11 min; a la tasa agregada del clúster (12.56 Mkeys/s), unos 2 min, y recorrer todo el espacio 36^6 unos 3 min. El caso real más costoso fue la clave de ocho caracteres `n3s4mmcv`: el cracker distribuido la recuperó tras 2.456×10^{12} candidatos (índice 2,456,777,905,475, ≈87% de 36^8) en 247,619 s ≈ 68.8 h a 9.92 Mkeys/s agregados. Cada carácter adicional multiplica por 36 ese costo.

7 Conclusión y trabajo diferido

Se recuperaron las claves de auth RIPv2/RFC 2082 de las seis capturas tras reconstruir la construcción keyed-MD5. Cada clave se validó contra todos los paquetes autenticados de su captura: 179/179 en total. Un clúster maestro–trabajador por demanda sobre TCP, con recuperación ante fallos por heartbeat y por desconexión, reproduce las mismas claves y es claramente CPU-bound, pues la coordinación añade una sobrecarga mínima frente al costo de hashear.

Recomendaciones. El costo de recuperación crece como el tamaño del alfabeto elevado a la longitud de la clave, por eso conviene usar claves largas sobre un alfabeto amplio y migrar de RIP a un protocolo con autenticación más fuerte.

Referencias

Anthropic (2026). Claude Code: asistencia en código y redacción. `https://www.anthropic.com/claude-code`.

Baker, F. and Atkinson, R. (1997). RIP-2 MD5 Authentication. Technical Report RFC 2082, Internet Engineering Task Force. `https://www.rfc-editor.org/rfc/rfc2082`.

Malkin, G. S. (1998). RIP Version 2. Technical Report RFC 2453, Internet Engineering Task Force. `https://www.rfc-editor.org/rfc/rfc2453`.

Apéndice A: Barrido completo de escalamiento

Host único, 10 núcleos lógicos, base-36 keyed-MD5, multiprocessing puro sin sockets. La tabla lista Mkeys/s agregados y por núcleo para 1 a 10 procesos; la tasa por núcleo cae cuando el número de procesos supera el presupuesto de núcleos físicos, por los núcleos de eficiencia y la contención del planificador. Datos fuente en `results/benchmarks.json`.

Procesos	Agregado (Mkeys/s)	Por núcleo (Mkeys/s)
1	1.99	1.99
2	3.94	1.97
4	7.09	1.77
6	8.89	1.48
8	10.67	1.33
10	10.95	1.09

Tabla 5: Barrido de escalamiento de BM2 (multiprocessing puro, sin sockets).

Análisis multimedia y código: Repo con video y código